# Q2: Data Layer — Database Schema, ORM Models & Pydantic Schemas

> ***Question:*** *Map out the database schema. Identify the core SQLAlchemy ORM models, explain the primary relationships (One-to-Many, Many-to-Many) between them, and show the corresponding Pydantic schemas used for request validation and response serialization.*

---

## Overview

The data layer is split across two files:

| File | Purpose |
|---|---|
| `app/models/models.py` | SQLAlchemy ORM — 22 table classes + 11 Python enums |
| `app/schemas/schemas.py` | Pydantic — request validation & response serialization |

All models inherit from `Base` (defined in `app/core/database.py`). Tables are **never auto-created** by the app — schema is managed exclusively by Alembic migrations.

---

## Enums

All enums are defined as `str, Enum` so they serialize naturally to/from JSON strings.

| Enum | Values |
|---|---|
| `UserRole` | `ADMIN`, `MANAGER`, `DRIVER` |
| `DriverStatus` | `ACTIVE`, `ON_TRIP`, `ON_BREAK`, `OFF_DUTY` |
| `VehicleStatus` | `FREE`, `ASSIGNED`, `EN_ROUTE`, `MAINTENANCE`, `OUT_OF_SERVICE` |
| `TripStatus` | `SCHEDULED`, `ACTIVE`, `COMPLETED`, `CANCELLED` |
| `TripDirection` | `OUTBOUND`, `INBOUND` |
| `ShiftType` | `MORNING`, `EVENING` |
| `RotationPosition` | `DRIVER_1`, `DRIVER_2`, `DRIVER_3` |
| `MaintenanceStatus` | `PENDING`, `APPROVED`, `IN_PROGRESS`, `COMPLETED`, `REJECTED` |

| Enum | Values |
| --- | --- |
| `MaintenanceType` | `REGULAR` , `EMERGENCY` |
| `ReplacementReason` | `BREAK` , `EMERGENCY_CROWDING` , `EMERGENCY_BREAKDOWN` , `NO_SHOW` |
| `TripTicketStatus` | `ISSUED` , `USED` , `CANCELLED` , `EXPIRED` |
| `IoTSensorType` | `ENGINE_TEMP` , `OIL_PRESSURE` , `BRAKE_PAD` , `BATTERY_VOLTAGE` |
| `RerouteStatus` | `PENDING` , `APPROVED` , `REJECTED` |
| `NotificationStatus` | `PENDING` , `DELIVERED` , `READ` |

---

# Core ORM Models & Relationships

## Relationship Map (Visual)

```
Garage
  ■■■ Driver (many)
  ■■■ Vehicle (many)

User ■■(1:1)■■ Driver
                ■■■ Trip (1:many)
                ■■■ BreakLog (1:many)
                ■■■ DriverExchange outgoing (1:many)
                ■■■ DriverExchange incoming (1:many)
                ■■■ RerouteLog (1:many)

Vehicle
  ■■■ Trip (1:many)
  ■■■ GPSTracking (1:many)
  ■■■ MaintenanceRequest (1:many)
  ■■■ IotSensorReading (1:many)
  ■■■ CameraReading (1:many)
  ■■■ CrowdingEvent (1:many)
  ■■■ GateLog (1:many, optional FK)

Route
  ■■■ RouteStop (1:many, ordered)
  ■■■ Trip (1:many)
  ■■■ RotationAssignment (1:many)

RotationAssignment ■■(FK)■■ Driver
                   ■■(FK)■■ Vehicle
                   ■■(FK)■■ Route
                   ■■■ DriverExchange (1:many)

Trip
  ■■■ Ticket (1:many)
  ■■■ CameraReading (1:many)
```

```
    ■■■ CrowdingEvent (1:many)
    ■■■ RerouteLog (1:many, optional)

  MaintenanceRequest
    ■■■ requested_by → User
    ■■■ approved_by  → User (nullable)
```

> ***Note:*** *There are no explicit Many-to-Many join tables. The* `RotationAssignment` *acts as a resolved junction between* `Driver` *,* `Vehicle` *, and* `Route` *per shift/date — it is a concrete business entity, not a pure join table.*

---

# Model-by-Model Reference

## `Garage`

**Table:** `garages`

Represents the physical depot.

| Column | Type | Notes |
|---|---|---|
| `id` | Integer PK | |
| `name` | String(255) | |
| `address` | String(500) | nullable |
| `latitude` / `longitude` | Float | nullable |
| `total_capacity` | Integer | default 50 |
| `current_occupancy` | Integer | default 0 |

**Relationships:** No back-populated relationships defined (drivers and vehicles hold a `garage_id` FK).

---

## `User`

**Table:** `users`

Authentication and role identity. Every human actor in the system is a `User` .

| Column | Type | Notes |
|---|---|---|
| `id` | Integer PK | |
| `email` | String(255) | unique |
| `hashed_password` | String(255) | bcrypt |
| `full_name` | String(255) | |

| Column | Type | Notes |
|---|---|---|
| `role` | `UserRole` enum | PG native enum |
| `phone` | String(20) | nullable |
| `preferred_language` | String(5) | default `"en"` |
| `is_active` | Boolean | default True |

**Relationships:**

```
driver: Optional["Driver"]  # One-to-One (uselist=False)
```

---

## `Driver`

**Table:** `drivers`

Driver profile — extends `User` with operational data. One `User` → one `Driver` (unique FK).

| Column | Type | Notes |
|---|---|---|
| `user_id` | FK → `users.id` | unique |
| `license_number` | String(50) | unique |
| `garage_id` | FK → `garages.id` | nullable |
| `status` | `DriverStatus` enum | |
| `current_vehicle_id` | FK → `vehicles.id` | nullable |
| `current_route_id` | FK → `routes.id` | nullable |
| `total_trips_today` / `total_trips_all_time` | Integer | |
| `break_time_remaining` | Float | minutes, default 60 |
| `total_break_time_today` | Float | |
| `trips_since_last_break` | Integer | |
| `fatigue_score` | Float | 0.0–100.0 |
| `current_shift` | `ShiftType` | nullable |

**Relationships:**

```
user: "User"                        # Many-to-One (back to User)
vehicle: Optional["Vehicle"]        # Many-to-One (current assignment)
trips: List["Trip"]                 # One-to-Many
```

---

## `Vehicle`

**Table:** `vehicles`

Bus inventory. Tracks real-time position and operational state.

| Column | Type | Notes |
|---|---|---|
| `plate_number` | String(20) | unique |
| `model` | String(100) | |
| `capacity` | Integer | default 50 — used for crowding % calc |
| `status` | `VehicleStatus` enum | |
| `garage_id` | FK → `garages.id` | |
| `current_latitude` / `current_longitude` | Float | updated by GPS hardware |
| `mileage` | Float | |
| `fuel_level` | Float | 0–100 |

**Relationships:**

```
current_driver: Optional["Driver"]              # One-to-One (current)
trips: List["Trip"]                             # One-to-Many
gps_logs: List["GPSTracking"]                   # One-to-Many
maintenance_requests: List["MaintenanceRequest"] # One-to-Many
```

---

## `Route` + `RouteStop`

**Tables:** `routes` , `route_stops`

`Route` defines a bus line. `RouteStop` holds the ordered waypoints.

`Route` **key columns:** `name` , `start_location` , `end_location` , `distance_km` , `estimated_time_minutes` , `fare` , `turnaround_time_minutes` , `is_active`

`RouteStop` **key columns:** `route_id` (FK), `stop_name` , `sequence_order` , `latitude` , `longitude` , `dwell_time_minutes`

**Constraint:** `UniqueConstraint("route_id", "sequence_order")` — no duplicate stop positions per route.

**Relationships on Route:**

```
stops: List["RouteStop"]                    # One-to-Many (ordered)
trips: List["Trip"]                         # One-to-Many
rotations: List["RotationAssignment"]       # One-to-Many
```

---

## `RotationAssignment`

**Table:** `rotation_assignments`

The daily shift plan. Assigns one `Driver` + one `Vehicle` to one `Route` at a specific `position` (DRIVER_1/2/3) for a given `shift_date` + `shift_type` .

**Unique constraint:** `(route_id, shift_type, position, shift_date)` — one assignment per slot.

| Column | Type | Notes |
|---|---|---|
| `route_id` | FK → `routes.id` | |
| `driver_id` | FK → `drivers.id` | |
| `vehicle_id` | FK → `vehicles.id` | |
| `shift_type` | `ShiftType` | MORNING / EVENING |
| `position` | `RotationPosition` | DRIVER_1 / 2 / 3 |
| `shift_date` | Date | |
| `shift_start_time` / `shift_end_time` | DateTime | |
| `is_active` | Boolean | |

**Relationships:**

```
route: "Route"
driver: "Driver"
vehicle: "Vehicle"
```

---

## `Trip`

**Table:** `trips`

A single bus journey (one direction on one route). The central operational entity.

| Column | Type | Notes |
|---|---|---|
| `driver_id` | FK → `drivers.id` | |
| `vehicle_id` | FK → `vehicles.id` | |
| `route_id` | FK → `routes.id` | |
| `rotation_assignment_id` | FK → `rotation_assignments.id` | nullable |
| `direction` | `TripDirection` | OUTBOUND / INBOUND |
| `status` | `TripStatus` | SCHEDULED → ACTIVE → COMPLETED |
| `scheduled_start` / `scheduled_end` | DateTime | planned times |

| Column | Type | Notes |
|---|---|---|
| `actual_start` / `actual_end` | DateTime | nullable |
| `passenger_count` | Integer | updated by camera hardware |
| `crowding_score` | Float | 0–100 |
| `is_crowded` | Boolean | set at ≥70% |
| `fare_collected` | Float | route.fare × passenger_count |
| `is_extra_dispatch` | Boolean | triggered by crowding ≥90% |
| `is_late` | Boolean | |

**Relationships:**

```
driver: "Driver"
vehicle: "Vehicle"
route: "Route"
tickets: List["Ticket"]    # One-to-Many
```

---

## `GPSTracking`

**Table:** `gps_tracking`

**Index:** `(vehicle_id, recorded_at)` for fast time-range queries.

Append-only log of GPS pings from hardware.

| Column | Type |
|---|---|
| `vehicle_id` | FK → `vehicles.id` |
| `trip_id` | FK → `trips.id` (nullable) |
| `latitude` / `longitude` | Float |
| `speed` / `heading` | Float |
| `recorded_at` | DateTime |

---

## `Ticket`

**Table:** `tickets`

Passenger ticket per trip. `ticket_code` auto-generated as 8-char hex token.

| Column | Type | Notes |
|---|---|---|
| `ticket_code` | String(8) | unique, auto `secrets.token_hex(4).upper()` |

| Column | Type | Notes |
|---|---|---|
| `trip_id` | FK → `trips.id` | |
| `passenger_name` | String(255) | nullable |
| `seat_number` | String(10) | nullable |
| `price` | Float | |
| `status` | `TripTicketStatus` | ISSUED / USED / CANCELLED / EXPIRED |
| `purchase_time` | DateTime | |
| `validation_time` | DateTime | nullable — set on scan |

---

### `MaintenanceRequest`

**Table:** `maintenance_requests`

Tracks vehicle service requests through a full approval workflow.

| Column | Type | Notes |
|---|---|---|
| `vehicle_id` | FK → `vehicles.id` | |
| `requested_by_id` | FK → `users.id` | |
| `approved_by_id` | FK → `users.id` | nullable |
| `type` | `MaintenanceType` | REGULAR / EMERGENCY |
| `status` | `MaintenanceStatus` | PENDING → APPROVED → IN_PROGRESS → COMPLETED |
| `priority` | Integer | 1 (highest) – 5 |
| `title` / `description` | String / Text | |
| `estimated_cost` / `actual_cost` | Float | nullable |
| `rejection_reason` | Text | nullable |

**Relationships:**

```
vehicle: "Vehicle"
requested_by: "User"   # foreign_keys=[requested_by_id]
approved_by: "User"    # foreign_keys=[approved_by_id]
```

*Two explicit `foreign_keys=` declarations are required here because `MaintenanceRequest` has **two FKs pointing to the same `users` table** — SQLAlchemy needs disambiguation.*

---

## `Notification`

**Table:** `notifications`

User alerts with a delivery lifecycle: `PENDING` → `DELIVERED` → `READ` .

| Column | Type |
|---|---|
| `user_id` | FK → `users.id` |
| `title` / `message` | String / Text |
| `notification_type` | String(50) |
| `status` | `NotificationStatus` |

---

## `DriverExchange`

**Table:** `driver_exchanges`

Records every driver swap event (break replacement, emergency crowding, breakdown, no-show).

| Column | Type |
|---|---|
| `rotation_assignment_id` | FK → `rotation_assignments.id` |
| `outgoing_driver_id` | FK → `drivers.id` |
| `incoming_driver_id` | FK → `drivers.id` |
| `reason` | `ReplacementReason` |
| `exchange_time` / `return_time` | DateTime |
| `trip_id` | FK → `trips.id` (nullable) |

---

## `BreakLog`

**Table:** `break_logs`

Immutable record of every break taken per driver per shift.

| Column | Type |
|---|---|
| `driver_id` | FK → `drivers.id` |
| `shift_date` | Date |
| `break_number` | Integer |
| `start_time` / `end_time` | DateTime |

| Column | Type |
|---|---|
| `duration_minutes` | Float (nullable) |
| `replaced_by_driver_id` | FK → `drivers.id` (nullable) |

---

### Hardware / IoT Models

| Model | Table | Purpose |
|---|---|---|
| `GateLog` | `gate_logs` | ANPR scan events: GRANTED / DENIED / IGNORED |
| `CameraReading` | `camera_readings` | In-cabin passenger count per trip |
| `IotSensorReading` | `iot_sensor_readings` | Engine/oil/brake/battery telemetry (indexed on `vehicle_id, recorded_at` ) |
| `CrowdingEvent` | `crowding_events` | Crowding incidents, `auto_dispatch_triggered` flag (indexed on `trip_id` ) |
| `GateCamera` | `gate_cameras` | Registry of ANPR cameras (IP, location, gate type) |

---

### Reporting / Audit Models

| Model | Table | Purpose |
|---|---|---|
| `DailyReport` | `daily_reports` | Daily aggregates: trips, revenue, crowding, on-time %. `unique=True` on `report_date` . |
| `AuditLog` | `audit_logs` | Every user action: `action` , `entity_type` , `entity_id` , `old_values` (JSON), `new_values` (JSON), `ip_address` |
| `RerouteLog` | `reroute_logs` | Driver reroute requests with approval state and two route FKs ( `original_route_id` , `new_route_id` ) |

---

## Pydantic Schemas

---

All schemas inherit from `BaseSchema` :

```
class BaseSchema(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

`from_attributes=True` enables direct construction from SQLAlchemy ORM instances.

---

## Schema Pattern Per Domain

Every domain follows the same four-schema pattern:

| Suffix | Used For | Key Trait |
|---|---|---|
| `*Base` | Shared fields | Common to create + response |
| `*Create` | `POST` body | May add validators |
| `*Update` | `PATCH` body | All fields `Optional` |
| `*Response` | API output | Adds `id` , timestamps, nested relations |

---

## Auth & User

```
# Input
class UserCreate(UserBase):
    password: str  # validated: 8+ chars, upper, lower, digit, special char

class SignupRequest:
    email: EmailStr
    full_name: str
    password: str   # same strength validator
    role: UserRole
    phone: Optional[str]  # regex: E.164 format

# Output
class UserResponse(UserBase):
    id: int
    created_at: datetime
    updated_at: datetime

# Tokens
class Token:
    access_token: str
    token_type: str
    refresh_token: Optional[str]

class PasswordChangeRequest:
    current_password: str
    new_password: str  # strength validated
```

**Password validation rule** (shared `_validate_password_strength()` ):

- Min 8 characters
- At least one uppercase, one lowercase, one digit, one special character ( `!@#$%^&*...` )

## Driver

```
class DriverCreate(DriverBase):
    user_id: Optional[int] = None
    user: Optional[UserCreate] = None  # supports nested user+driver creation in one request

class DriverResponse(DriverBase):
    id: int
    user: Optional[UserResponse]       # nested User embedded in response
    status: DriverStatus
    total_trips_today: int
    break_time_remaining: float
    fatigue_score: float  # not in DriverBase — only in response
    ...
```

## Vehicle

```
class VehicleBase:
    plate_number: str = Field(..., pattern=r'^[A-Z0-9-]{3,10}$')  # enforced format
    capacity: int = Field(50, gt=0)

class VehicleUpdate:
    status: Optional[VehicleStatus]
    current_latitude: Optional[float]   # updated by GPS hardware
    current_longitude: Optional[float]
    mileage: Optional[float]
    fuel_level: Optional[float]
```

## Route & Stops

```
class RouteCreate(RouteBase):
    stops: List[RouteStopCreate] = []   # stops embedded in route creation

class RouteResponse(RouteBase):
    id: int
    stops: List[RouteStopResponse] = [] # stops embedded in response
```

Route creation and retrieval both carry the full stop list — no separate stop endpoint needed.

## Trip

```
class TripResponse(TripBase):
    status: TripStatus
    passenger_count: int
    crowding_score: float
    fare_collected: float
    is_late: bool
    route: Optional[RouteResponse]      # nested — used for driver trip detail view
    vehicle: Optional[VehicleResponse]  # nested
```

## Hardware Schemas (no `BaseSchema` — raw `BaseModel`)

These are inbound-only; no ORM output needed:

```
class GPSSubmit:
    latitude: float
    longitude: float
    speed: Optional[float] = 0.0
    heading: Optional[float] = 0.0
    trip_id: Optional[int] = None

class IoTSensorData:
    vehicle_id: int
    engine_temp_c: float
    oil_pressure_psi: float
    brake_pad_mm: float
    battery_voltage: float

class ANPRData:
    plate_number: str
    confidence: float
    gate_id: str

class CameraData:
    trip_id: int
    passenger_count: int
```

---

## Composite Schema

```
class UserWithDriverCreate:
    user: UserCreate
    driver: DriverBase
```

Allows creating a `User` + `Driver` in a single atomic request. The router handles the two-step DB insertion.

---

## Dashboard / Aggregate Schemas

```
class AdminDashboardStats:
    total_vehicles: int
    total_drivers: int
    total_routes: int
    total_users: int
    pending_maintenance: int
    active_trips: int
    trips_per_route: List[Dict[str, Any]]  # chart data

class ManagerDashboardStats:
    trips_today: int
    total_revenue: float
    on_time_percentage: float
    crowding_alerts: int
    pending_maintenance: int
```

---

# Key Design Decisions

| Decision | Reason |
|---|---|
| `create_type=False` on most SAEnums | Alembic migrations already created the PG enum types — letting SQLAlchemy try to recreate them on startup would fail |
| `Ticket.status` stored as `String(20)` not SAEnum | DB enum only has 3 values but Python enum has 4 ( `CANCELLED` ) — avoids a migration to patch |
| Two `foreign_keys=` on `MaintenanceRequest` | Both `requested_by_id` and `approved_by_id` point to `users` — SQLAlchemy requires explicit disambiguation |
| `RotationAssignment` unique constraint on `(route_id, shift_type, position, shift_date)` | Prevents double-booking a slot; enforced at DB level, not just application logic |
| `GPSTracking` and `IotSensorReading` have composite indexes on `(vehicle_id, recorded_at)` | These are high-volume append-only tables; the index makes time-range queries per vehicle fast |
| `Ticket.ticket_code` auto-generated via `secrets.token_hex(4).upper()` | 8-char cryptographically random code, unique index enforced |

---

*Generated: 2026-04-16*